

ATmega32A Module

Seven-Segment Display

Ali Sahafi

1 Theory

A seven-segment display is simply **eight LEDs** (segments *a–g* plus the decimal point) arranged as a digit. In a **common-anode** display all LED anodes share the VCC pin, so a segment lights up when its cathode pin is pulled **LOW** — the segments are *active-low*. Displaying “5” just means turning on the right subset of segments (a, f, g, c, d).

This driver is hardwired to **PORTB**: one call writes the complete segment pattern for a digit.

2 Pin mapping

The driver expects the display wired to PORTB as follows (each segment through a $330\,\Omega$ resistor):

PORTB bit	PB0	PB1	PB2	PB3	PB4	PB5	PB6	PB7
Segment	e	d	c	b	f	g	a	dp

A bit value of **0** turns **the segment on** (active-low, common-anode). For example digit 8 is `0b10000000`: every segment on, decimal point off.

3 API reference

Method	Description
<code>SevenSeg(digit)</code>	Display digit 0–9 on PORTB; any other value shows E

The function also *returns* the segment pattern it wrote, and PORTB must be configured as output first: `GPIO.setDirection(DDRB, ALL, OUTPUT);`

4 Example: counting 0–9

Wiring.

- Common-anode 7-segment display: common pin to VCC, segments to PB0–PB7 via $330\,\Omega$ resistors (mapping table above).

Build & flash. `make sevensseg`

```
/*
 * Seven-Segment Example -- count 0 to 9 forever
 *
 * Wiring:
```

```

*   - Common-anode 7-segment display on PORTB (segments via 330R resistors)
*   - The display is hardwired to PORTB in the driver
*
* Build & flash: make sevenseg
*/

#define F_CPU 8000000UL
#include "../gpio/gpio.hpp"
#include "sevenseg.hpp"

int main() {
    GPIO.setDirection(DDRB, ALL, OUTPUT); // 7-segment lives on PORTB

    while (true) {
        for (uint8_t digit = 0; digit <= 9; digit++) {
            SevenSeg(digit);
            _delay_ms(500);
        }
    }

    return 0;
}

```

5 Exercises

1. Combine this module with GPIO: count one step per *button press* instead of every 500 ms.
2. Extend `SevenSeg()` with hexadecimal digits A–F. Derive the segment patterns yourself using the pin-mapping table.
3. What changes if your display is **common-cathode** instead of common-anode? Modify the driver accordingly (hint: one operator).